



ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

Graph Neural Networks

Prof. Gianluca Moro,
Tutor: Dr. Giacomo Frisoni

DISI, University of Bologna, Cesena
Via dell'Università, 50 I-47522 Cesena (FC), Italy
{name.surname}@unibo.it

Graph Neural Networks - Moro G., Frisoni G.

1

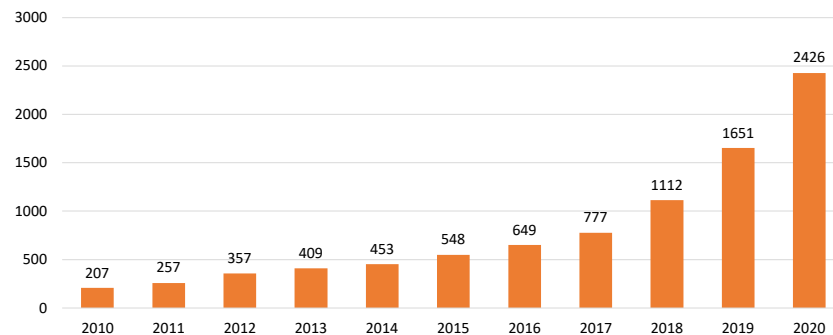
Outline

- **Graph data** 
- Graph types
- Graph representation techniques
- Graph representation learning
- Graph neural networks
- Popular architectures
- How to train a graph neural network

Why graphs ?

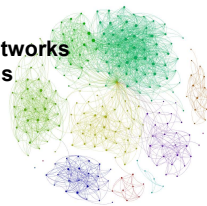
- Graphs are powerful mathematical abstractions, a general language for describing and analyzing entities with relations
- A lot of real-world data does not «live» on grids

Number of ArXiv papers with “graph” in the title

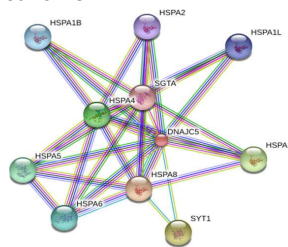


Graphs are ubiquitous – i

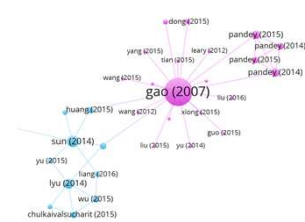
- Social networks
- Communication networks
- Multi-agent systems



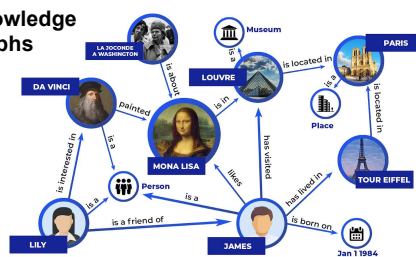
- Protein interaction networks



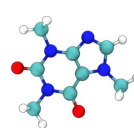
- Citation networks



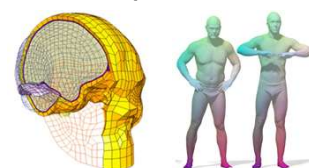
- Knowledge graphs



- Molecules

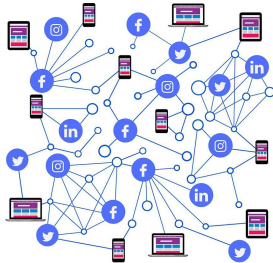


- Volumetric meshes
- Surface meshes
- 3D shapes

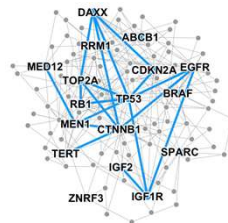


Graphs are ubiquitous – ii

- **Recommender systems**



- **Disease pathways**
(protein interactions that collectively produce some disease phenotype)

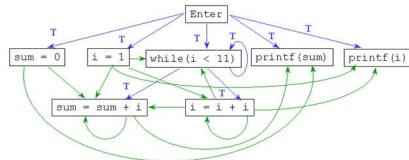


- **Underground networks**

- **Traffic networks**



- **Code graphs** (e.g., program dependence graphs)



- **Computer networks**

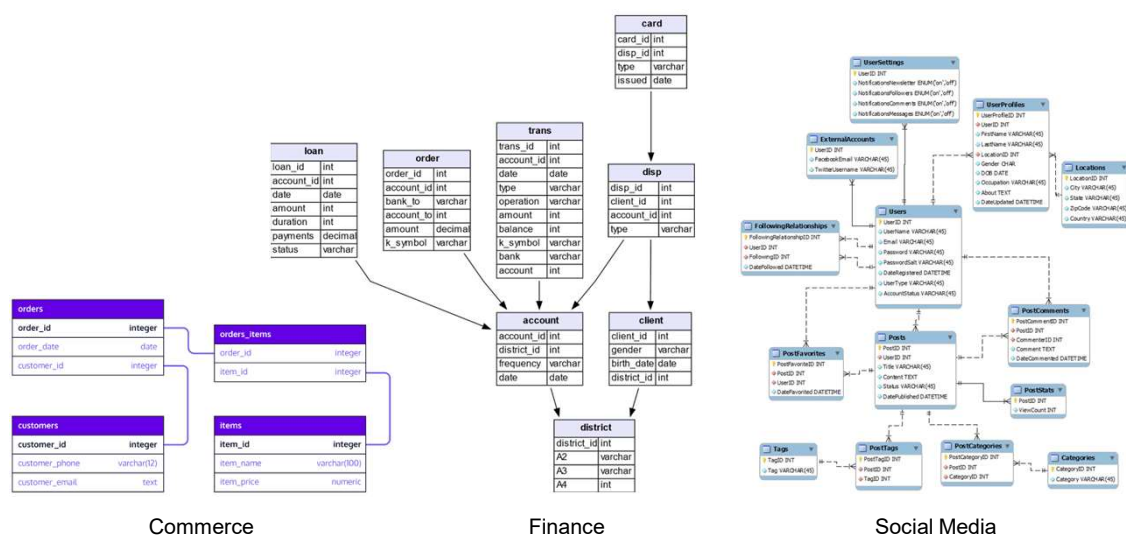


disi-unibo-nlp.github.io

Graph Neural Networks - Moro G., Frisoni G.

5

Graphs are ubiquitous... Even in databases

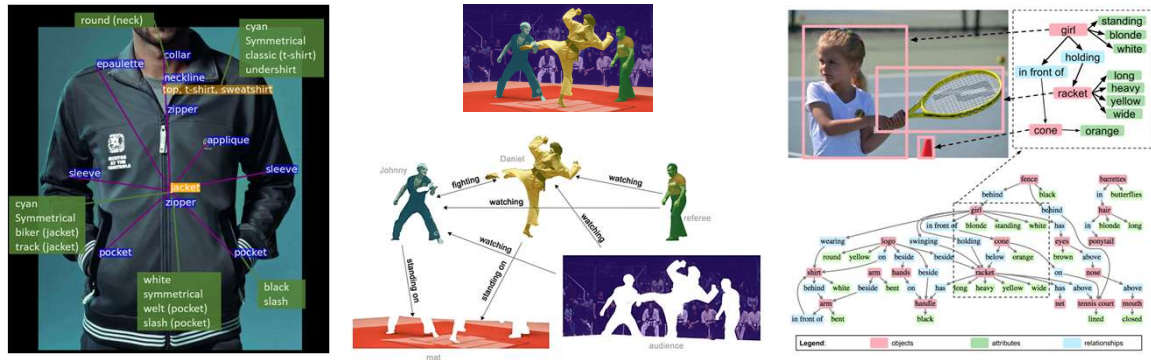


disi-unibo-nlp.github.io

Graph Neural Networks - Moro G., Frisoni G.

6

Graphs are ubiquitous... Even in images



Visual segmentation and scene graphs
(parse an image into a semantic graph expressing the visual relationships)
E.g., [FashionPedia](#)

Graphs are ubiquitous... Even in text documents – i

- Token sequence**

Graphs → are → all → around → us

- Co-occurrence graph**

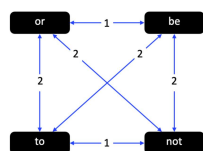
(how many times each token is surrounded by another token, considering a $\pm k$ window)

Text input: To be, or not to be: ...

Co-occurrence matrix

	to	be	or	not
to		2	2	1
be	2		1	2
or	2	1		1
not	1	2	1	

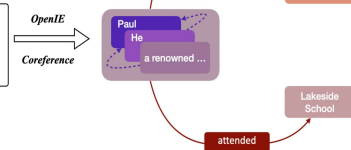
Co-occurrence graph



E.g., $k = 3$

- Extracted triplets + co-reference resolution**

Text input: Paul, a renowned computer scientist, grew up in Seattle. He attended Lakeside School.



- Topic graphs (topic modeling)**

There was the \$5 million Deutsche Bank Championship to prepare for and the Ryder Cup is a few weeks away ...

A sport psychologist says how footballers should prepare themselves for ...

Dolphin groups rely on socialites to keep them from collapsing, scientists claim

Sports

Business

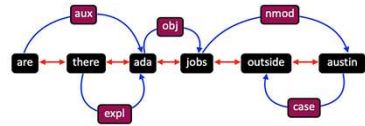
Research

Graphs are ubiquitous... Even in text documents – ii

Syntactic parsing

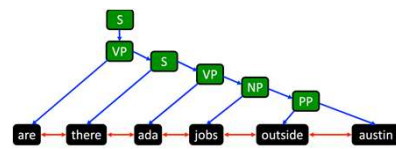
Grammatical relationships with words as nodes

- **Dependency parsing graph**
(word-to-word relationships)



E.g., *aux* = auxiliary verbs, *expl* = expletive, *obj* = object, *nmod* = nominal modifier, *case* = preposition

- **Constituency parsing graph**
(hierarchical phrase structure)

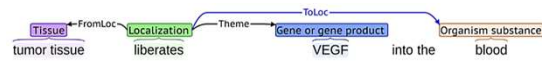


E.g., *S* = sentence, *VP* = verb phrase, *NP* = noun phrase, *PP* = prepositional phrase

Semantic parsing

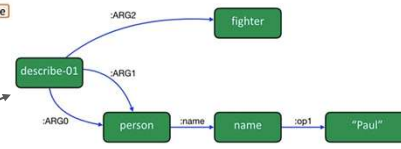
Decoupling concept units from language competencies

- **Events**



- **Abstract Meaning Representations**

«Paul's description of himself: a Fighter»

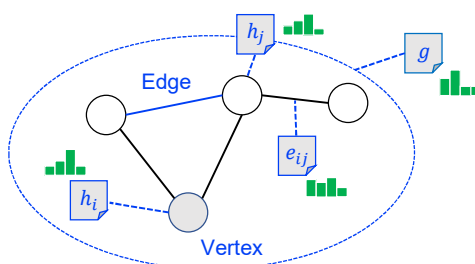


Outline

- Graph data
- Graph types 
- Graph representation techniques
- Graph representation learning
- Graph neural networks
- Popular architectures
- How to train a graph neural network

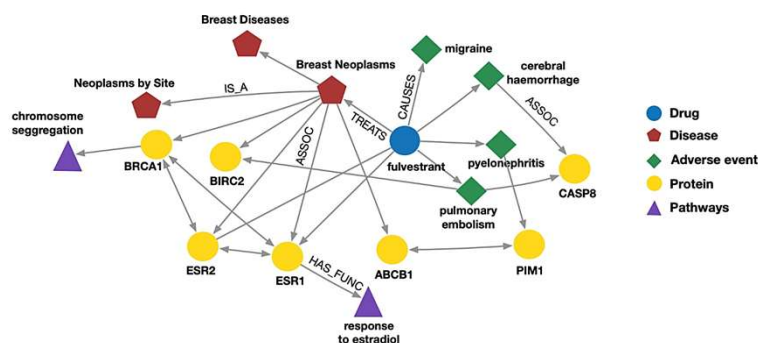
Computational graph structure

- $G = (V, E)$
 - V is the set of nodes (also known as vertices); $E \subseteq V \times V$ is the set of edges
- Features (attribute vectors with pre-defined size)
 - Node features : $h_i, h_j \in \mathbb{R}^{d_v}$ (e.g., atom type)
 - Edge features : $e_{ij} \in \mathbb{R}^{d_e}$ (e.g., bound type)
 - Graph features : $g \in \mathbb{R}^{d_g}$ (e.g., molecule energy, number of nodes, longest path)



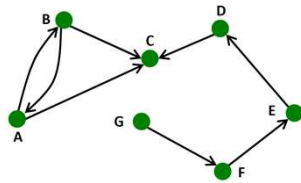
Heterogeneous graphs

- Presence of different node types and potentially different edge types
- $G = (V, E, T_V, T_E)$
 - T_V is a set of node types such that each node $v \in V$ is assigned a mapping $\phi(v) : V \rightarrow T_V$
 - T_E is a set of edge types such that each edge $e \in E$ is assigned a mapping $\psi(e) : E \rightarrow T_E$
 - $|T_V| > 1, |T_E| > 1$

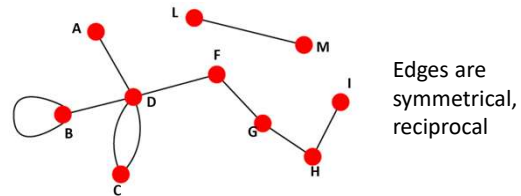


Direction and weights for edges

- In a directed graph (or digraph), E is a set of **ordered node pairs**

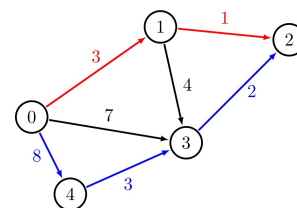


- In an undirected graph, E is a set of **unordered node pairs**



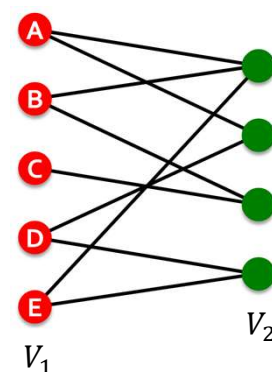
- $G = (V, E, w)$

- Weight function $w : E \rightarrow \mathbb{R}$
- E.g., in a road network graph, the weight could represent the distance or travel time between two cities



Bipartite graphs

- Type of graph in which the set of nodes can be divided into two disjoint sets such that no two nodes within the same set are adjacent
- $V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$
 - In other words, there are no edges between nodes within V_1 (and the same for V_2)
- Additional properties
 - It does not contain any odd-length cycles
 - V_1 and V_2 are sometimes called *partitions* or *color classes*, as a bipartite graph is often 2-colorable: each node can be assigned one of two colors such that no two adjacent nodes share the same color
- Examples
 - Authors-to-Papers, Actors-to-Movies, Users-to-Movies, Recipes-to-Ingredients



Outline

- Graph data
- Graph types
- **Graph representation techniques** 
- Graph representation learning
- Graph neural networks
- Popular architectures
- How to train a graph neural network

Graph representation: adjacency matrix – i

- The adjacency matrix A shows **the neighborhood of each node**

$$A \in \{0,1\}^{|V| \times |V|} \text{ where } A_{ij} = \begin{cases} 1 & \text{if there is an edge between node } i \text{ and node } j \\ 0 & \text{if there is no edge between node } i \text{ and node } j \end{cases}$$

▪ Properties

- If there are no self-loops, the diagonal elements are all 0
- For a directed graph, A is asymmetric; for an undirected graph, A is symmetric
- For a weighted graph, the entries in A represent the edge weights, $A \in \mathbb{R}^{|V| \times |V|}$
- Easily tensorizable (compatible with neural networks expecting grid-like arrays)

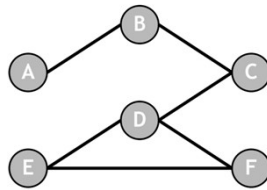
Time complexity

- **Edge lookup:** $O(1)$, it involves simply checking the value at the matrix position A_{ij}
- **Neighbor lookup:** $O(|V|)$, finding all neighbors of a node requires scanning a row of the matrix

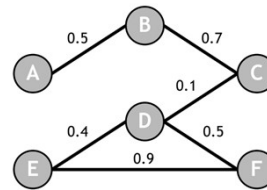
Space complexity

- $O(|V|^2)$, if the graph is sparse, the adjacency matrix becomes space-inefficient: in this case, most entries will be zero, which results in wasted space

Graph representation: adjacency matrix – ii



	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	0	0	0
C	0	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0



	A	B	C	D	E	F
A	0	0.5	0	0	0	0
B	0.5	0	0.7	0	0	0
C	0	0.7	0	0.1	0	0
D	0	0	0.1	0	0.4	0.5
E	0	0	0	0.4	0	0.9
F	0	0	0	0.5	0.9	0

Graph representation: degree matrix

- The diagonal matrix D shows the degree (# connections) of each node

$$D \in \mathbb{R}^{|V| \times |V|} \text{ where } D_{ij} = \begin{cases} \deg(v_i) & \text{if } i=j \\ 0 & \text{if } i \neq j \end{cases}$$

Sum of the weights of all edges incident to v_i : $\sum_{j=1}^n A_{ij}$

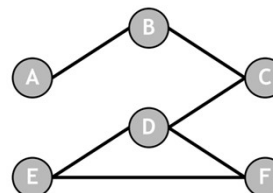
- Preferable when only degree information is required, or in conjunction with **spectral graph techniques** where space efficiency is a priority
 - Reveal insights about the **graph structure** (e.g., communities) that may not be obvious from the graph's original connections

Time complexity

- Degree lookup: $O(1)$

Space complexity

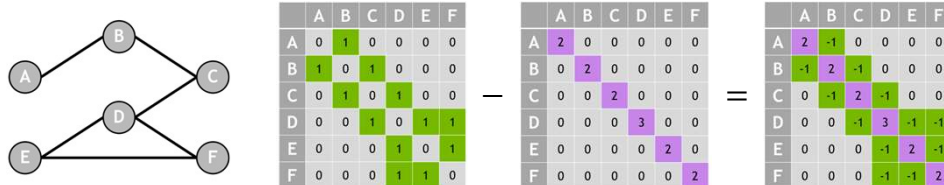
- Since D is diagonal, it can be efficiently stored as a 1-dimensional array, reducing complexity to $O(|V|)$



	A	B	C	D	E	F
A	2	0	0	0	0	0
B	0	2	0	0	0	0
C	0	0	2	0	0	0
D	0	0	0	3	0	0
E	0	0	0	0	2	0
F	0	0	0	0	0	2

Smoothness of a Graph: Laplacian Matrix Representation – i

- The Laplacian matrix L is defined as $D - A$ (D degree and A adjacency Matrices)
 - It indicates [the smoothness of a graph](#), i.e., how connected or cohesive it is (whether there is a path between every pair of nodes and how resistant the graph is to breaking into parts)
 - Practical uses in clustering, segmentation, and connectivity analysis (L is unnormalized)



- Properties
 - If G is undirected, then L is symmetric, meaning $L = L^T$
 - L is positive semi-definite, so all its eigenvalues are non-negative
 - The smallest eigenvalue of L is always 0
 - The number of zero eigenvalues corresponds to the number of connected components in G

Graph representation: laplacian matrix – ii

- Variant #1: normalized Laplacian matrix
 - $L_{norm} = I - D^{-1/2} A D^{-1/2}$
 - It better clusters graphs with large disparities in node degrees, such as social networks with a few highly connected hubs and many low-degree nodes
 - It ensures that the result is not dominated by high-degree nodes
- Variant #2: random walk Laplacian matrix
 - $L_{rw} = I - D^{-1} A$
 - It models the transition probabilities from one node to another, normalized by node degrees
 - By focusing on transitions and steady states, it is helpful for capturing flow-like processes

Graph representation: edge list

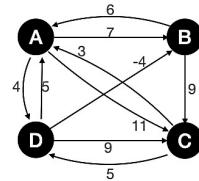
- An edge list is an unordered collection of node tuples, one for each edge
 - $\{(v_i, v_j) | (v_i, v_j) \in E\}$
 - For an undirected graph, $(v_i, v_j) = (v_j, v_i)$
 - For a weighted graph, $\{(v_i, v_j, w(v_i, v_j)) | (v_i, v_j) \in E\}$

Time complexity

- Edge lookup: $O(|E|)$
- Neighbor lookup: $O(|E|)$

Space complexity

- $O(|E|)$; efficient for sparse graphs where $|E| \ll |V|^2$;
inefficient for dense graphs where $|E| \approx |V|^2$



(from_node, to_node, edge weight)

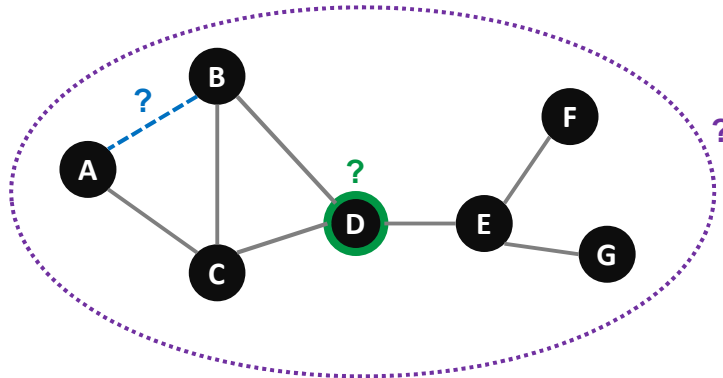
[(A, B, 7), (A, C, 3), (A, D, 4),
(B, A, 7), (B, C, 9), (B, D, -4),
(C, A, 3), (C, B, 9), (C, D, 5),
(D, A, 4), (D, B, -4), (D, C, 5)]

Outline

- Graph data
- Graph types
- Graph representation techniques
- **Graph representation learning** 
- Graph neural networks
- Popular architectures
- How to train a graph neural network

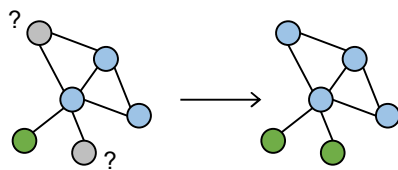
What can we learn on a graph ?

- Three main prediction types: **node-level**, **edge-level**, **graph-level**

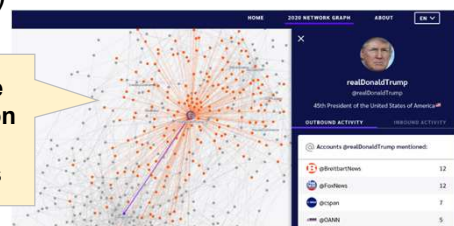


Node-level tasks

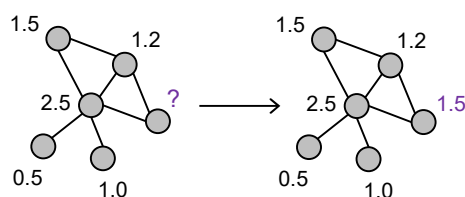
- Node classification (discrete outcome variables)



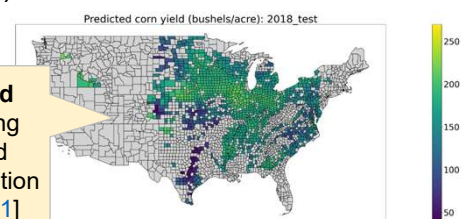
E.g., predict the political affiliation of a user from a graph of X posts



- Node regression (continuous outcome variables)



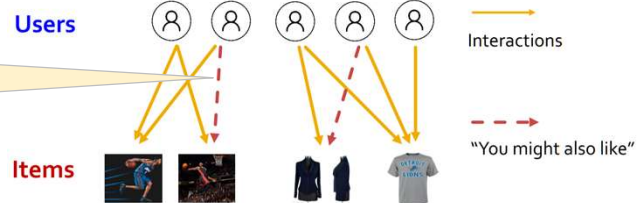
E.g., crop yield prediction using geospatial and temporal information [Fan et al., 2021]



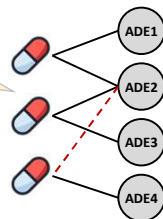
Edge-level tasks - i

- Link prediction: predict new, missing, or unknown edges

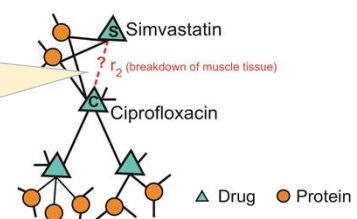
E.g., **recommend items users might like**
(watch movies, buy merchandise, listen to music...)



E.g., **predict adverse side effects of drugs**



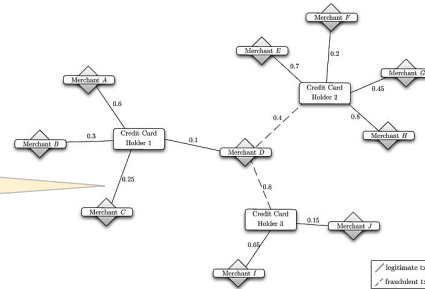
How likely will Simvastatin and Ciprofloxacin, when taken together, break down muscle tissue?



Edge-level tasks - ii

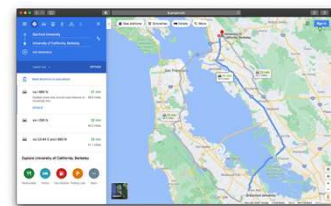
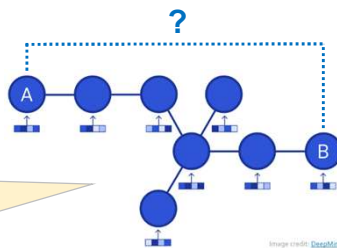
- Edge classification

E.g., **predict fraudulent transactions**
(nodes = merchants and card holders;
edge = transactions; weights = recency)



- Edge regression

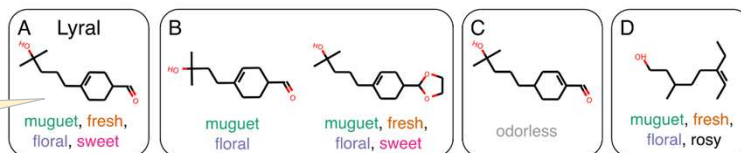
E.g., **estimate time to travel from A to B**
(nodes = road segments,
edges = connectivity
between road segments)



Graph-level tasks

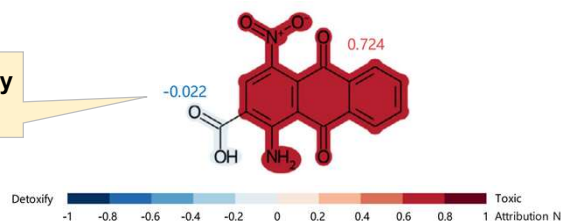
- Graph or subgraph classification

E.g., predict the smell of a molecule



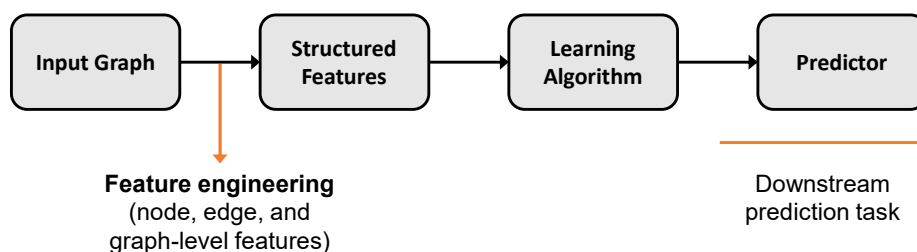
- Graph or subgraph regression

E.g., predict the toxicity rating of a molecule



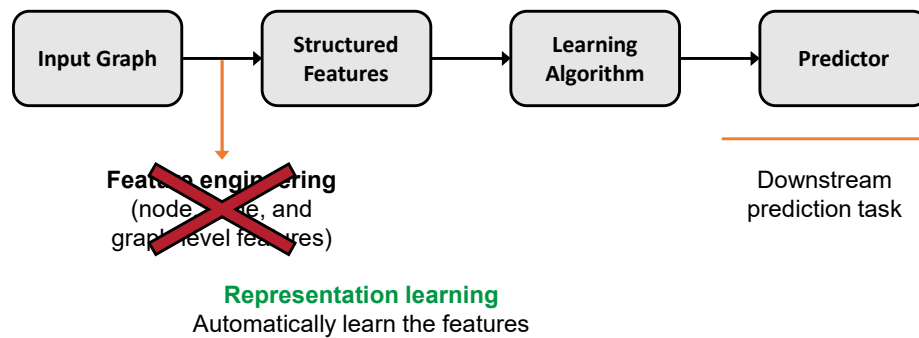
Recap: traditional machine learning

- Given an input graph G , extract features
- Then, learn a model (SVM, neural network, etc.) that maps features to labels



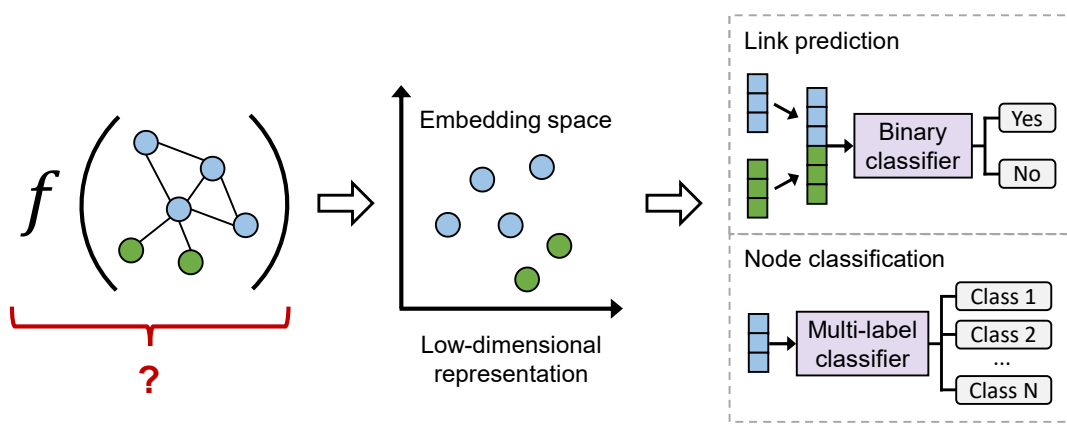
Recap: representation learning

- Given an input graph G , extract features
- Then, learn a model (SVM, neural network, etc.) that maps features to labels



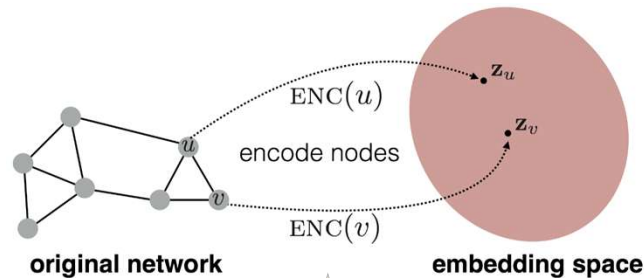
Graph representation learning

- Find a mapping function φ from a graph G to a d -dimensional vector $\varphi(G) \in \mathbb{R}^d$, preserving important graph properties such as structure and semantics



Node embeddings

- Encode nodes so that similarity in the embedding space (e.g., dot product) approximates similarity in the graph



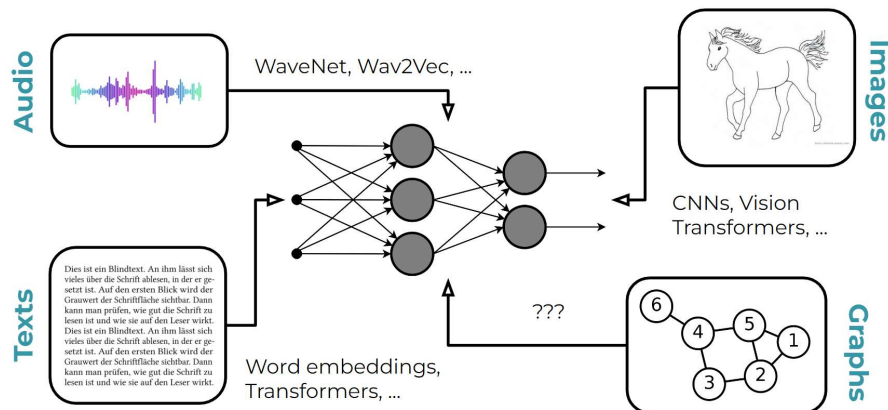
Today's solutions implement the function **ENC** with **Deep Graph Encoders** (multiple layers of non-linear transformations based on graph structure)

Outline

- Graph data
- Graph types
- Graph representation techniques
- Graph representation learning
- **Graph neural networks** 
- Popular architectures
- How to train a graph neural network

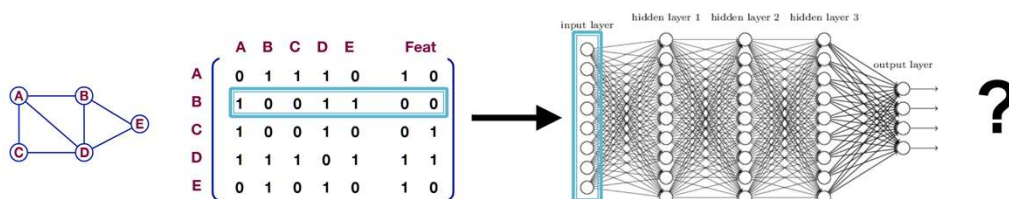
How can we learn on a graph?

- Modern deep learning toolbox is designed for sequences and grids
 - How do we take advantage of relational structure ?



A naïve approach

- Join adjacency matrix and **features** («Feat» in the last 2 matrix columns)
- Feed the joint representation of each node into a multi-layer perceptron (MLP)

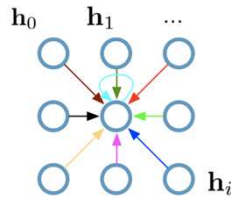
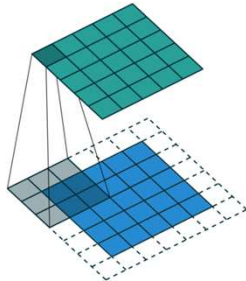


Issues with this idea

- $O(|V|)$ parameters:** the size of the first layer's weight matrix must match the input size; the parameters in the network grow significantly as the number of nodes increases
- Not applicable to graphs of different sizes:** neural networks expect fixed-size input, but **real-world graphs often vary in size**
- Sensitive to node ordering:** permuting the order of nodes changes the matrix's structure, even though the graph itself remains the same ← **we will come back to this point soon**

Recap: convolutional neural networks (CNNs) on grids

Single CNN layer
with 3x3 filter:



Update for a single pixel:

- Transform messages individually $W_i h_i$
- Add everything up $\sum_i W_i h_i$

Note: Neighbors are determined by the filter size

$h_i \in \mathbb{R}^d$ are the feature vectors (hidden representations) of a pixel/node i in the current layer

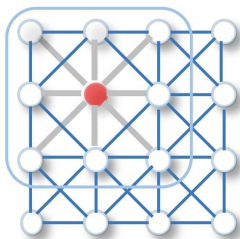
Full update (weighted average of the pixel values of the center node along with its neighbors):

$$h_i^{(l+1)} = \sigma \left(W_0^{(l)} h_0^{(l)} + W_1^{(l)} h_1^{(l)} + \dots + W_8^{(l)} h_8^{(l)} \right)$$

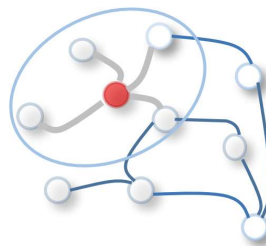
💡 **Generalize convolutions beyond simple lattices and leverage node features**

- An image can be considered as a fully connected graph
- Like CNNs, efficient learning on graphs relies on designing local operations with shared weights

2D convolution vs. graph convolution



- ☒ Locality (neighborhood)
- ☒ Fixed number of neighbors
- ☒ Neighbors are ordered



There is no fixed notion of **locality** or **sliding window** on a graph

- ☒ Locality (neighborhood)
- ☒ **Variable number of neighbors** (graphs vary in size and topology)
- ☒ **Neighbors are unordered** (information about node identity and ordering across multiple samples is rarely available)

No canonical node ordering

- **Observation:** we can have many different node orderings of the same graph



For a graph with $|V|$ nodes,
there are $|V|!$ different orderings

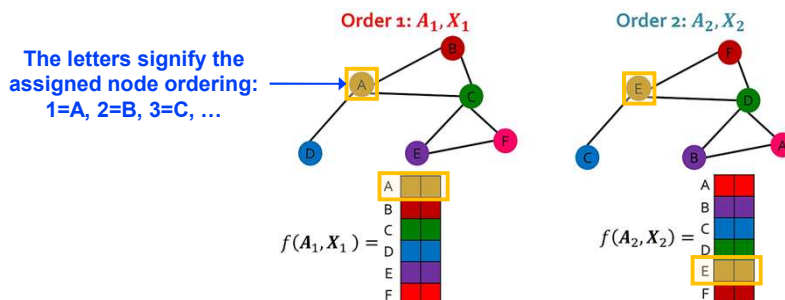
What we want:
different node orderings should
not affect how we understand or
process the graph

Example. 24 adjacency matrices for the same 4-nodes graph.

1:A,B,C,D 2:A,B,D,C 3:A,C,B,D 4:A,C,D,B 5:A,D,B,C 6:A,D,C,B 7:B,A,C,D 8:B,A,D,C 9:B,C,A,D 10:B,C,D,A 11:B,D,A,C 12:B,D,C,A 13:C,A,B,D 14:C,A,D,B 15:C,B,A,D 16:C,B,D,A 17:C,D,A,B 18:C,D,B,A 19:D,A,B,C 20:D,A,C,B 21:D,B,A,C 22:D,B,C,A 23:D,C,A,B 24:D,C,B,A

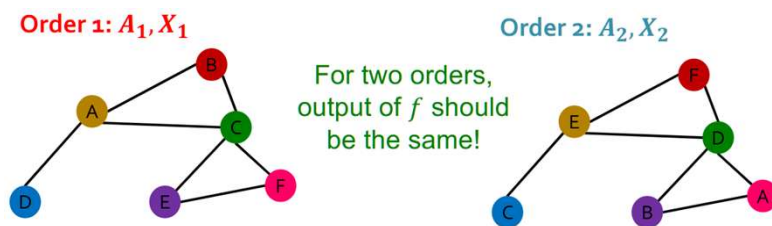
Permutation equivariance

- **When required:** node representation
 - Learn a function f that maps nodes of a graph $G = (A, X)$ to a matrix $\mathbb{R}^{|V| \times d}$
 - A = adjacency matrix, X = node feature matrix
- **Definition:** we say f is permutation equivariant if the output vector of a node at the same position in the graph remains unchanged for any ordering
 - $f(PAP^T, PX) = Pf(A, X)$ for any permutation P , i.e., a shuffle of the node order



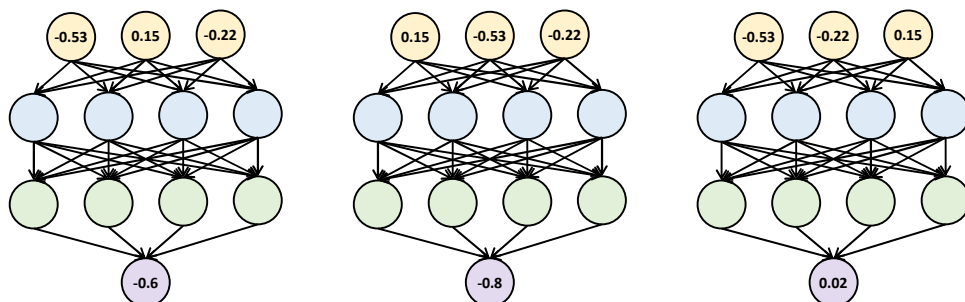
Permutation invariance

- **When required:** global or subgraph pooling operations
 - Learn a function f that maps a graph $G = (A, X)$ to a vector $\mathbb{R}^d$
- **Definition:** we say f is permutation invariant if $f(A_i, X_i) = f(A_j, X_j)$ for any ordering i and j
 - $f(PAP^T, PX) = f(A, X)$ for any permutation P



Two needed properties – i

- Is a **MLP** permutation invariant / equivariant ?
 - **No.**

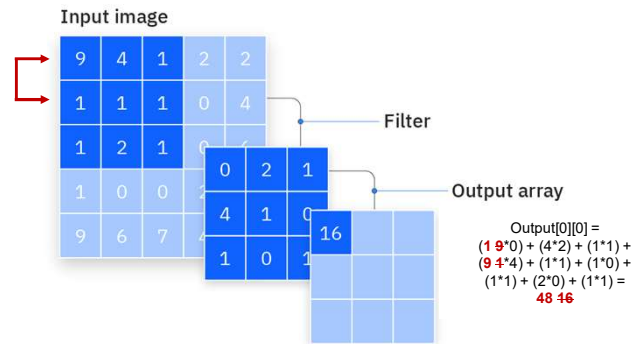


Switching the order of the inputs leads to different outputs.
A neural network modeling a graph should be permutation invariant and equivariant.

Two needed properties – ii

- Is a CNN permutation invariant / equivariant ?

• No.

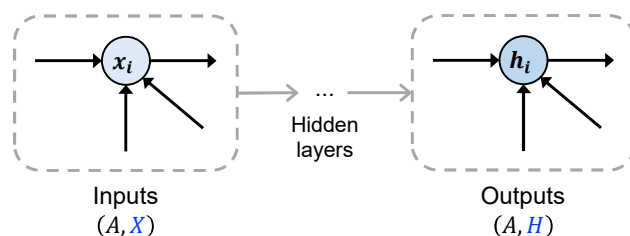


Switching the order of pixels leads to different outputs.

A neural network modeling a graph should be permutation invariant and equivariant.

Graph neural networks (GNNs): base idea

- A GNN is a **deep learning encoder model** that **automatically extracts relevant features for a graph**, using **permutation equivariant / invariant operations**
 - Starting with initial node feature vectors, it progressively transforms them based on their neighbors' feature vectors to get better representations
- GNNs are versatile building blocks, whose output can be coupled with any predictor to solve specific tasks



Z_i **Node classification**
 $Z_i = f(h_i)$

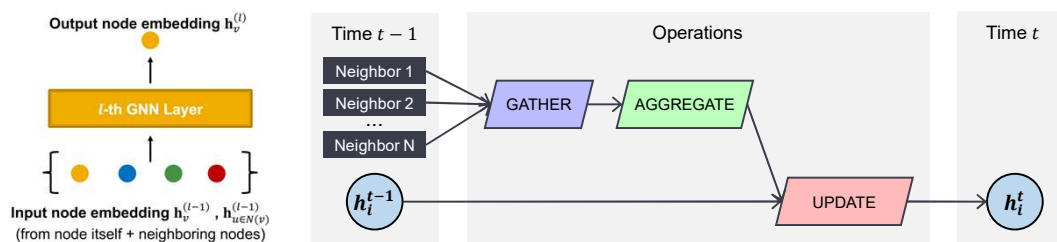
Z_G **Graph classification**
 $Z_G = f\left(\sum_i h_i\right)$

Z_{ij} **Link classification**
 $Z_{ij} = f(h_i, h_j)$

...

Anatomy of a single GNN layer

- **Message passing** and **neighborhood aggregation**: nodes exchange information with their neighbors to update their embeddings
- For each node, the process involves three operations
 1. **Gather**: Collects messages (embeddings) from neighboring nodes
 2. **Aggregate**: Combines all collected messages into a single fixed-size representation; must be permutation invariant (order of neighbors should not matter)
 3. **Update**: Creates new node embedding using aggregated information and current state; learnable update function (usually a MLP)



disi-unibo-nlp.github.io

Graph Neural Networks - Moro G., Frisoni G.

46

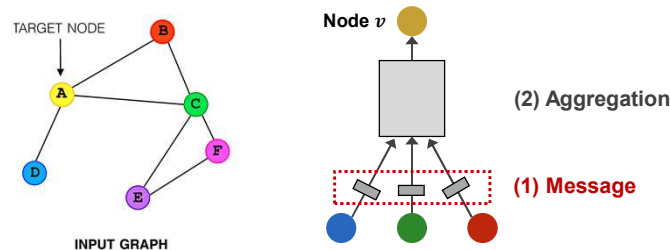
Single GNN layer: message computation

- **Intuitive metaphor**: each node creates and send a message to other nodes
- **Message function**:

$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)}(\mathbf{h}_u^{(l-1)})$$

▪ Example

- A linear layer $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$ ← Multiply node features with weight matrix $\mathbf{W}^{(l)}$



disi-unibo-nlp.github.io

Graph Neural Networks - Moro G., Frisoni G.

47

Single GNN layer: message aggregation and state update – i

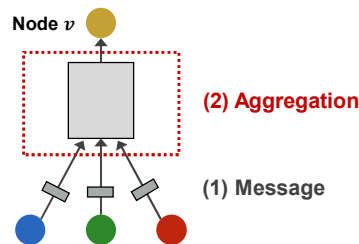
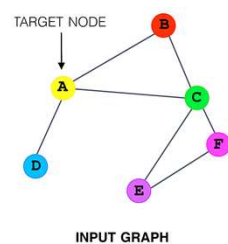
- **Intuition:** node v aggregates the messages from its neighbors u

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$$

- **Example**

- Sum($\cdot$), Mean($\cdot$), or Max($\cdot$) aggregator
- $\mathbf{h}_v^{(l)} = \text{Sum} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$

- **Mean:** normalized view (useful when nodes have a highly-variable number of neighbors)
- **Max:** highlight single salient features in local neighborhoods
- **Sum:** balance between mean and max (commonly used)



Single GNN layer: message aggregation and state update – ii

- **Issue:** information from node v itself could get lost
 - Computation of $\mathbf{h}_v^{(l)}$ does not directly depend on $\mathbf{h}_v^{(l-1)}$
- **Solution:** include $\mathbf{h}_v^{(l-1)}$ when computing $\mathbf{h}_v^{(l)}$
 - **(1) Message:** compute message from node v itself; usually, a different message computation is performed

$$\text{Blue, Green, Red} \quad \mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \qquad \text{Yellow} \quad \mathbf{m}_v^{(l)} = \mathbf{B}^{(l)} \mathbf{h}_v^{(l-1)}$$

- **(2) Aggregation:** after aggregating from neighbors, we can aggregate the message from node v itself; e.g., via concatenation or summation

First, aggregate from neighbors

$$\mathbf{h}_v^{(l)} = \text{CONCAT} \left(\text{AGG} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right), \mathbf{m}_v^{(l)} \right)$$

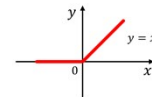
Then, aggregate the node itself

Single GNN layer: non-linearity

- Non-linearity – denoted as $\sigma(\cdot)$ – can be added to **message** or **aggregation**
 - Adds expressiveness
- Examples of activations applied to the i -th dimension of embedding x

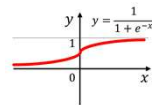
- Rectified linear unit (ReLU)**

- $\text{ReLU}(x_i) = \max(x_i, 0)$
 - Most commonly used



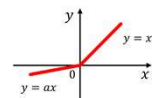
- Sigmoid**

- $\sigma(x_i) = \frac{1}{1+e^{-x_i}}$
 - Used only when you want to restrict the range of your embeddings



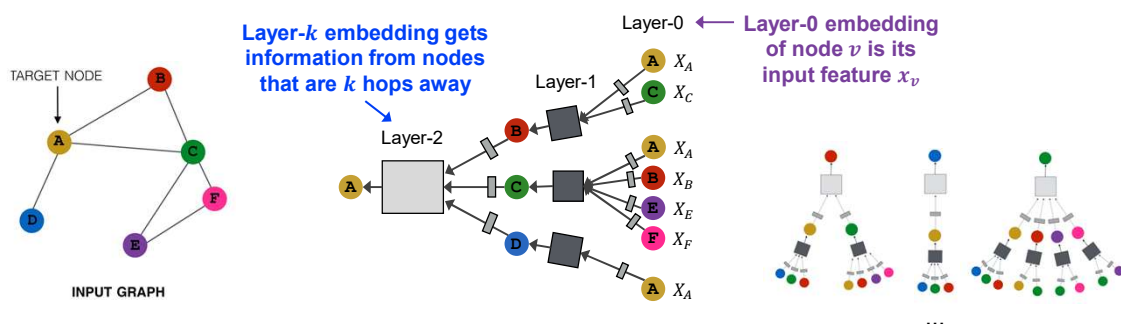
- Parameter ReLU**

- $\text{PReLU}(x_i) = \max(x_i, 0) + \alpha_i \min(x_i, 0)$
 - Where α_i is a trainable parameter
 - Empirically performs better than ReLU



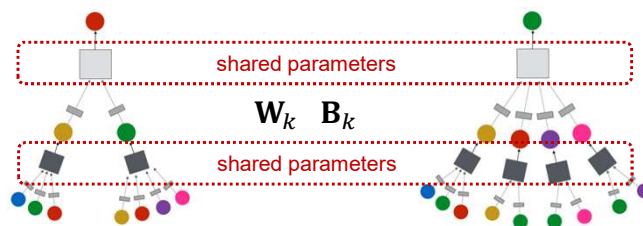
More GNN layers

- By stacking multiple propagation layers, a node can indirectly leverage the information about nodes farther away in the graph (**spread of local information**)
 - GNN depth = # propagation layers
- Every node defines a **computation graph** based on its neighborhood



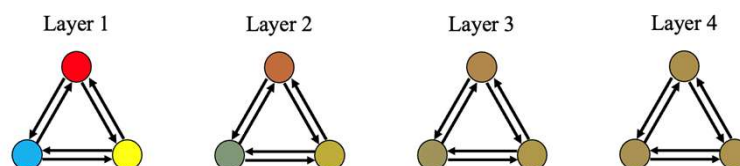
More GNN layers: shared parameters

- Same neural network weights are applied to each node's neighborhood
- Why ?
 - Enables learning generalizable node representations
 - Reduces model complexity
 - Allows processing graphs of different sizes
 - Guarantees permutation equivariance / invariance



More GNN layers: warning

- Performing multiple updates **increases the «receptive field»** of each node
- **Oversmoothing**
 - Interacting **nodes converge to indistinguishable representations** as the number of layers increases, diminishing the ability to capture important local properties and patterns
 - It occurs **when the number of layers is much greater than the problem radius**, i.e., the farther distance (graph hops) that information needs to travel to solve the task effectively



Example. The problem radius is 1 since the graph is fully connected.
The node representations already become difficult to distinguish by layer 3.

Why GNNs are permutation equivariant and invariant ?

▪ Permutation equivariance (node-level)

- The **aggregation function** is designed to be **symmetric**, i.e., the neighbors are treated as an **unordered set**; e.g., $\text{sum}(B, C, D) = \text{sum}(D, B, C)$
- Because the aggregation function is order-invariant for any single node's neighborhood, the entire GNN layer becomes permutation equivariant
- The representation for Node A (whatever its new label is) will still be computed from its neighbors (whatever their new labels are)

▪ Permutation invariant (graph-level)

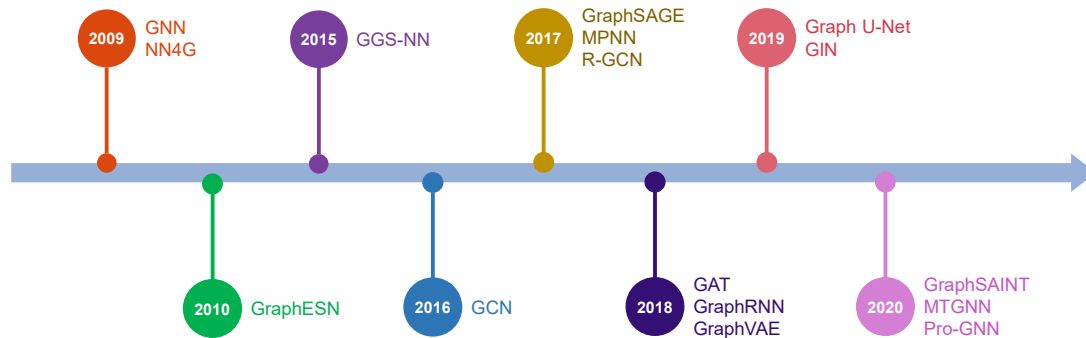
- To get a single vector for the whole graph, we apply a **global pooling function** on the final set of all node embeddings that is *also symmetric*
- This is often just a sum, mean or max operation across all node embeddings in the graph ← **we will come back to this point soon**

Outline

- Graph data
- Graph types
- Graph representation techniques
- Graph representation learning
- Graph neural networks
- **Popular architectures** 
- How to train a graph neural network

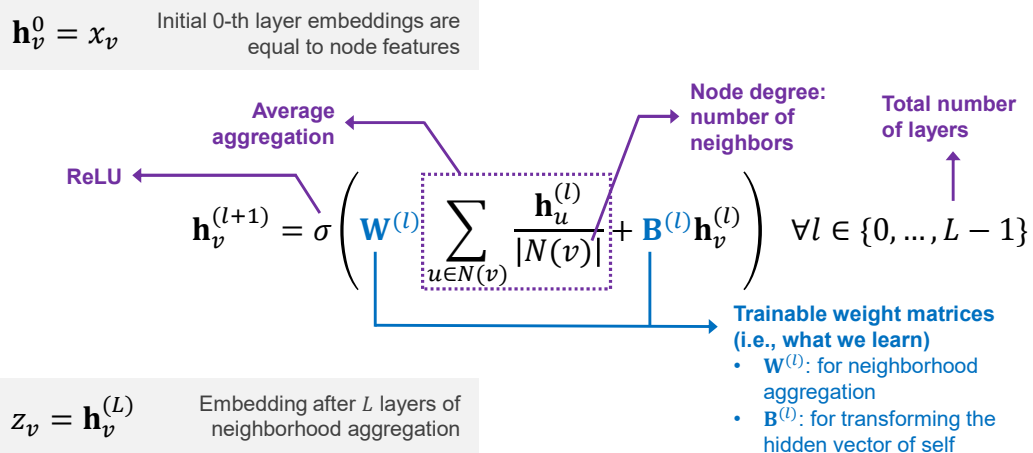
GNNs: a huge family

- Set of **mathematical models** aimed at processing information from nodes and combining it with that of their neighbors as they progress through the layers
 - The way information is processed and combined (i.e., gather, aggregate, update functions are implemented) determines the overall **architecture**



Graph convolutional networks (GCNs) [Kipf and Welling, 2016]

- The most classic GNN layer



GCNs: matrix formulation

- Let $H^{(l)} = [h_1^{(l)} \dots h_{|V|}^{(l)}]^T$
 - Then, $A_v H^{(l)} = \sum_{u \in N(v)} h_u^{(l)}$ ← Implements message passing with a summation aggregation
 A_v = the v -th row of the adjacency matrix
- Let D be the degree matrix where $D_{v,v} = \text{Deg}(v) = |N(v)|$
 - The inverse of D , D^{-1} , is also diagonal: $D_{v,v}^{-1} = 1/|N(v)|$
- Therefore...

$$\sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \quad \Rightarrow \quad D^{-1} A H^{(l)}$$

Neighborhood aggregation

$$H^{(l+1)} = \sigma(\tilde{A} H^{(l)} W_l^T + H^{(l)} B_l^T)$$

Where $\tilde{A} = D^{-1} A$ **Self-transformation**

- Note:** not all GNNs can be expressed in a simple matrix form, when aggregation function is complex

GraphSAGE [Hamilton et al., 2017]

(2) Further aggregate over the node itself

(1) Aggregate from node neighbors

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\mathbf{W}^{(l)} \text{CONCAT} \left(\mathbf{h}_v^{(l)}, \text{AGG} \left(\{ \mathbf{h}_u^{(l)}, \forall u \in N(v) \} \right) \right) \right)$$

Optional ℓ_2 **normalization**

$$\mathbf{h}_v^{(l+1)} \leftarrow \frac{\mathbf{h}_v^{(l+1)}}{\|\mathbf{h}_v^{(l+1)}\|_2}, \forall v \in V$$

It ensures that the magnitudes of the node embeddings remain bounded

Three options

- Average**

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l)}}{|N(v)|}$$
- Transform neighbor vectors and pool**

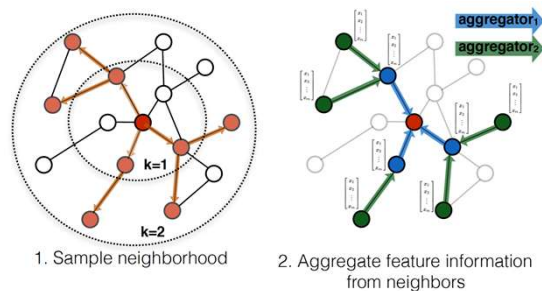
$$\text{AGG} = \text{Mean}(\{ \text{MLP}(\mathbf{h}_u^{(l)}), \forall u \in N(v) \})$$
- Apply LSTM to reshuffled neighbors**

$$\text{AGG} = \text{LSTM}(\{ \mathbf{h}_u^{(l)}, \forall u \in \pi(N(v)) \})$$

The selection depends on the dataset and the specific task, but the MLP-based pooling is generally found to perform the best

GraphSAGE: neighbor sampling

- When graphs are large and dense, it can be unfeasible to perform aggregations over all neighbors for each node, as the edge number becomes quadratic in $|V|$
- GraphSAGE samples a fixed number of neighbors at multiple distances and treat them as direct neighbors of node v
 - E.g., 10 neighbors for the 2nd hop, 5 neighbors for the 1st hop



Graph attention networks (GATs) [Veličković et al., 2018]

- Motivations

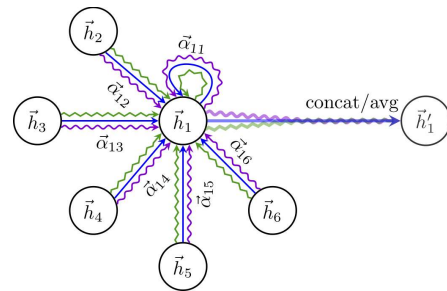
$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{m}_u^{(l)} \right)$$

↓

- In GCN and GraphSAGE (w/ average aggregation)
 - $\alpha_{vu} = \frac{1}{|N(v)|}$ is the **weighting factor (importance)** of node u 's message to node v
 - $\Rightarrow \alpha_{vu}$ is defined **explicitly** based on the structural properties of the graph (**node degree**)
 - $\Rightarrow$ For a given node v , all its neighbors $u \in N(v)$ receive the same weight $1/|N(v)|$
 - $\Rightarrow$ All neighbors $u \in N(v)$ are equally important to node v
- However, not all node's neighbors are equally important
 - Social network example: To predict your interests, your **close friends** are far more influential than a **random celebrity** you follow

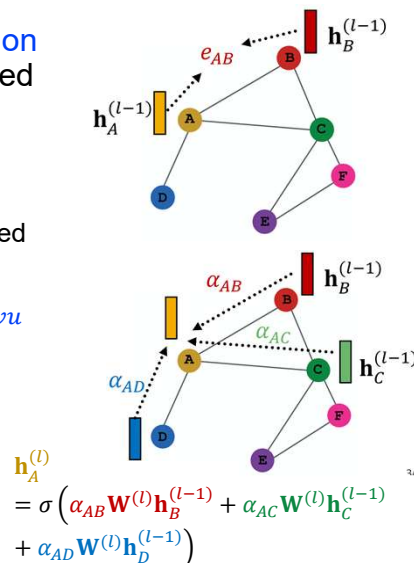
GATs: make the neighbor importance learnable

- **Goal:** specify **arbitrary importance** to different neighbors of each node in G
 - Can weighting factors $\alpha_{v,u}$ be learned ?
- **Idea:** compute the embedding $\mathbf{h}_v^{(l)}$ of each node following an **attention strategy**
 - The attention focuses on the important parts of the input data and fades out the rest
 - The GNN should devote more computing power on that small but important part of the data
 - Which part of the data is more important depends on the context and is learned
- **Attention on graphs**
 - Nodes attend over their neighborhoods' message
 - **Implicitly** specifying different weights to different nodes in a neighborhood



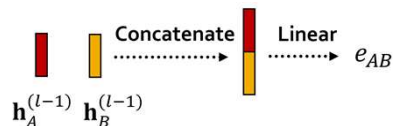
GATs: attention mechanism - i

1. Let an **attention mechanism** a compute **attention coefficients** e_{vu} across pairs of nodes u, v based on their messages:
 - $e_{vu} = a(\mathbf{m}_u^{(l)}, \mathbf{m}_v^{(l)})$
 - $e_{vu} \rightarrow$ importance of u 's message to node v
 - ⚡ Attention coefficient computation can be parallelized across all edges of the graph
2. **Normalize** e_{vu} into the **final attention weight** α_{vu}
 - Use the softmax function, so that $\sum_{u \in N(v)} \alpha_{vu} = 1$
 - $\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$
3. **Weighted sum** based on the final attention weight α_{vu}
 - $\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right)$



GATs: attention mechanism – ii

- We still must answer one question: **what is the attention mechanism α ?**
- The GAT approach is agnostic to the choice of α
 - E.g., use a simple single-layer neural network
 - α trainable parameters = weights in the linear layer



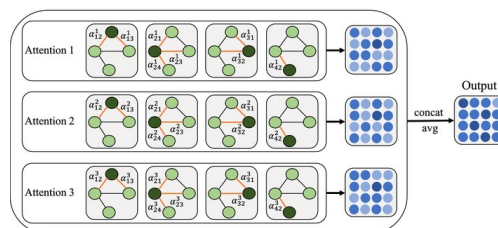
$$e_{AB} = \alpha \left(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} \right)$$

$$= \text{Linear} \left(\text{Concat} \left(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} \right) \right)$$

- Parameters α are trained jointly with the other parameters of the neural net $\mathbf{W}^{(l)}$
 - End-to-end learning

GATs: multi-head attention

- Multi-head attention **stabilizes the learning process** of attention mechanism
- Create multiple attention scores (each replica with a different set of parameters)
 - $\mathbf{h}_v^{(l)}[1] = \sigma \left(\sum_{u \in N(v)} \alpha_{vu}^1 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right)$
 - $\mathbf{h}_v^{(l)}[2] = \sigma \left(\sum_{u \in N(v)} \alpha_{vu}^2 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right)$
 - $\mathbf{h}_v^{(l)}[3] = \sigma \left(\sum_{u \in N(v)} \alpha_{vu}^3 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right)$
- Outputs are aggregated by concatenation or summation
 - $\mathbf{h}_v^{(l)} = \text{AGG} \left(\mathbf{h}_v^{(l)}[1], \mathbf{h}_v^{(l)}[2], \mathbf{h}_v^{(l)}[3] \right)$

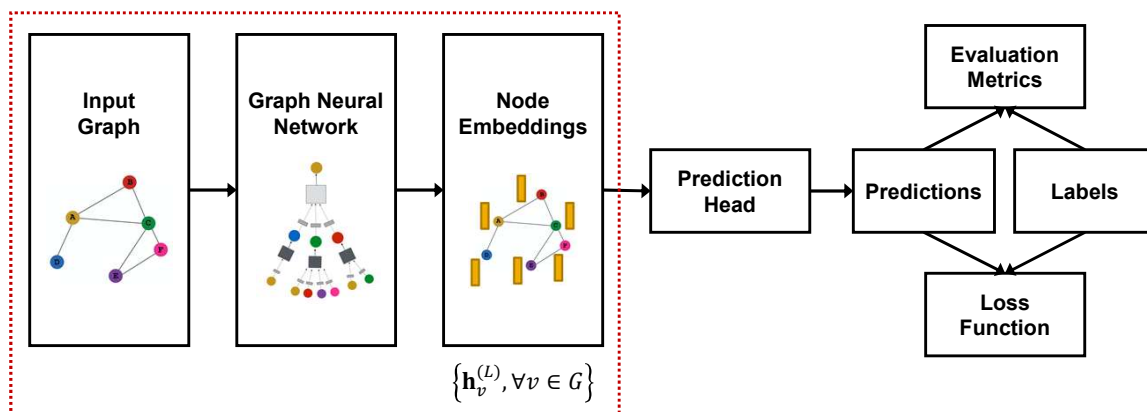


Outline

- Graph data
- Graph types
- Graph representation techniques
- Graph representation learning
- Graph neural networks
- Popular architectures
- **How to train a graph neural network** 

GNN training pipeline

So far what we have covered



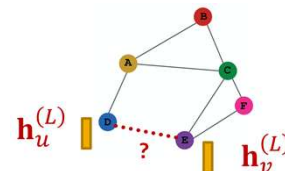
Prediction heads: node-level

- We can directly make prediction using node embeddings
- After GNN computation, we have d -dim node embeddings: $\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\}$
- Suppose we want to make c -way prediction
 - Classification: classify among c categories
 - Regression: regress on c targets
- $\hat{\mathbf{y}}_v = \text{Head}_{\text{node}}(\mathbf{h}_v^{(L)}) = \mathbf{W}^{(H)} \mathbf{h}_v^{(L)}$
 - $\mathbf{W}^{(H)} \in \mathbb{R}^{c \times d}$
 - We map node embeddings from $\mathbf{h}_v^{(L)} \in \mathbb{R}^d$ to $\hat{\mathbf{y}}_v \in \mathbb{R}^c$ so that we can compute the loss

Prediction heads: edge-level

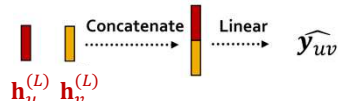
- Make prediction using pairs of node embeddings

$$\hat{\mathbf{y}}_{uv} = \text{Head}_{\text{edge}}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)})$$



▪ (1) Concatenation + Linear

- We have seen this in graph attention



- $\hat{\mathbf{y}}_{uv} = \text{Linear}(\text{Concat}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)}))$
- Here, $\text{Linear}(\cdot)$ will map $2d$ -dimensional embeddings to c -dim embeddings (c -way prediction)

▪ (2) Dot product

- $\hat{\mathbf{y}}_{uv} = (\mathbf{h}_u^{(L)})^T \mathbf{h}_v^{(L)}$
- This approach only applies to 1-way prediction (e.g., link prediction: predict the existence of an edge)
- It can be extended to c -way prediction

$$\hat{\mathbf{y}}_{uv}^{(1)} = (\mathbf{h}_u^{(L)})^T \mathbf{W}^{(1)} \mathbf{h}_v^{(L)} \quad \hat{\mathbf{y}}_{uv}^{(c)} = (\mathbf{h}_u^{(L)})^T \mathbf{W}^{(c)} \mathbf{h}_v^{(L)}$$

$$\hat{\mathbf{y}}_{uv} = \text{Concat}(\hat{\mathbf{y}}_{uv}^{(1)}, \dots, \hat{\mathbf{y}}_{uv}^{(c)}) \in \mathbb{R}^c$$

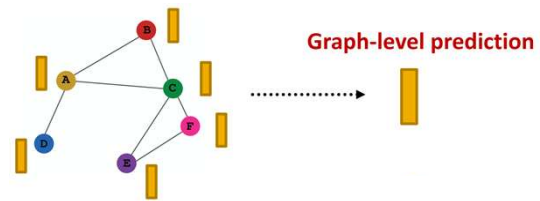
Prediction heads: graph-level – i

- Make prediction using all the node embeddings in our graph

$$\hat{\mathbf{y}}_G = \text{Head}_{\text{graph}} \left(\left\{ \mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G \right\} \right)$$

(1) Global pooling

- $\hat{\mathbf{y}}_G = \text{Mean} \left(\left\{ \mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G \right\} \right)$
- $\hat{\mathbf{y}}_G = \text{Max} \left(\left\{ \mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G \right\} \right)$
- $\hat{\mathbf{y}}_G = \text{Sum} \left(\left\{ \mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G \right\} \right)$
- These options work great for small graphs



Issue: global pooling over a (large) graph will lose information

- Node embeddings for G_1 : $\{-1, -2, 0, 1, 2\}$
- Node embeddings for G_2 : $\{-10, -20, 0, 10, 20\}$
- Clearly, G_1 and G_2 have very different node embeddings: their structure should be different
- If we do global sum pooling...
 - G_1 : $\hat{\mathbf{y}}_G = \text{Sum}(\{-1, -2, 0, 1, 2\}) = 0$
 - G_2 : $\hat{\mathbf{y}}_G = \text{Sum}(\{-10, -20, 0, 10, 20\}) = 0$
 - We cannot differentiate G_1 and G_2**

Prediction heads: graph-level – ii

(2) Hierarchical Global Pooling

- Aggregate all the node embeddings hierarchically

Toy example: we will aggregate via $\text{ReLU}(\text{Sum}(\cdot))$

- We first separately aggregate the first 2 nodes and the last 3 nodes
- Then we aggregate again to make the final prediction

G_1 node embeddings: $\{-1, -2, 0, 1, 2\}$

- Round 1:** $\hat{\mathbf{y}}_a = \text{ReLU}(\text{Sum}(\{-1, -2\})) = 0$, $\hat{\mathbf{y}}_b = \text{ReLU}(\text{Sum}(\{0, 1, 2\})) = 3$
- Round 2:** $\hat{\mathbf{y}}_G = \text{ReLU}(\text{Sum}(\{\hat{\mathbf{y}}_a, \hat{\mathbf{y}}_b\})) = 3$

G_2 node embeddings: $\{-10, -20, 0, 10, 20\}$

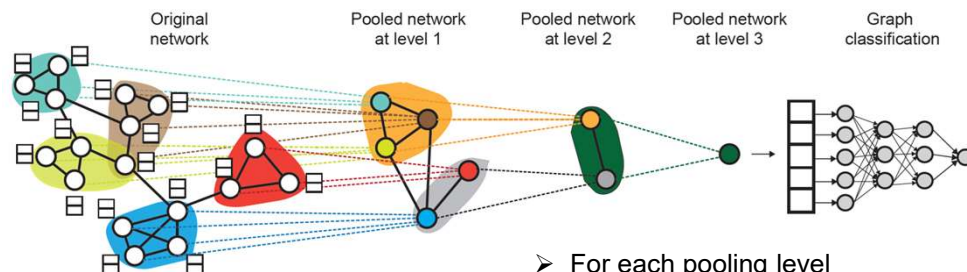
- Round 1:** $\hat{\mathbf{y}}_a = \text{ReLU}(\text{Sum}(\{-10, -20\})) = 0$, $\hat{\mathbf{y}}_b = \text{ReLU}(\text{Sum}(\{0, 10, 20\})) = 30$
- Round 2:** $\hat{\mathbf{y}}_G = \text{ReLU}(\text{Sum}(\{\hat{\mathbf{y}}_a, \hat{\mathbf{y}}_b\})) = 30$

Now we can differentiate G_1 and G_2

Prediction heads: graph-level – iii

▪ (2) Hierarchical Global Pooling

- DiffPool [Ying et al., 2018]



- Leverage two independent GNNs (jointly trained)
 - **GNN A**: compute node embeddings
 - **GNN B**: compute the cluster that a node belongs to
 - GNNs **A** and **B** can be executed in parallel

- For each pooling level
 - Use clustering assignments from **GNN B** to aggregate node embeddings produced by **GNN A**
 - Create a single new node for each cluster, maintaining edges between clusters to generate a new pooled network

Supervised vs Self-supervised

▪ Supervised learning on graphs

- Labels come from external sources
- **Node labels** y_v : in a citation network, which subject area does a node belong to
- **Edge labels** y_{vu} : in a transaction network, whether an edge is fraudulent
- **Graph labels** y_G : among molecular graphs, the drug likeness (i.e., likely candidate)
- **Advice**: reduce your tasks to node / edge / graph labels, since they are easy to work with
 - E.g., some nodes form a cluster; we can treat the cluster that a node belongs to as a node label

▪ Self-supervised learning on graphs

- **Problem**: sometimes we only have a graph, without any external labels
- **Solution**: find supervision signals within the graph itself
- **Node labels** y_v : node statistics, such as clustering coefficient, PageRank, ...
- **Edge labels** y_{vu} : link prediction, i.e., hide the edge between two nodes, predict if there should be a link
- **Graph labels** y_G : graph statistics, such as predicting if two graphs are isomorphic

Notation clarification:

We will use $\hat{y}^{(i)}$ to refer the prediction and $y^{(i)}$ to refer the target for the i -th data point

Classification loss

- E.g., node classification: which category does a node belong to
- **Cross entropy (CE)** is very common
 - C -way prediction for i -th data point, with C being the number of classes



One-hot label

$$\text{CE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) = - \sum_{j=1}^C y_j^{(i)} \log(\hat{y}_j^{(i)})$$



Prediction after Softmax($\cdot$)

i -th data point
 j -th class

- Total loss over all N training examples

$$\text{Loss} = \sum_{i=1}^N \text{CE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$

Intuition: the lower the loss, the closer the prediction is to one-hot

Regression loss

- E.g., predict the likeness of a molecular graph being a viable drug candidate
- **Mean squared error (MSE)** a.k.a. L2 loss is very popular
 - C -way regression for i -th data point



Real valued vector of targets

$$\text{MSE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) = \sum_{j=1}^C (y_j^{(i)} - \hat{y}_j^{(i)})^2$$



Real valued vector of predictions

- Total loss over all N training examples: $\text{Loss} = \sum_{i=1}^N \text{MSE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$



Thanks for the attention.
Is all you need ... for the moment